

2.2.2 Top-Down Parser (LL (1))

- It uses LMD and is equivalent to DFS in graph.

2.2.3 Algorithm to construct Parsing Table

1. Remove Left Recursion if any.
 2. Left Factor [Remove Common Prefix]
 3. Find first and follow set
 4. Construct the table.
- If we increase the look ahead symbol:

2.2.4 Removal of common Prefix: (Left Factor)

1. $S \rightarrow a \mid a b \mid a A$ $S \rightarrow a Y$
 $Y \rightarrow \epsilon \mid b \mid A$
2. $A \rightarrow a b A \mid a A \mid b$
 $A \rightarrow a \times \mid b$ $A \rightarrow a \times \mid b$
 $X \rightarrow b A \mid A$ $X \rightarrow b A \mid a x \mid b$
 $A \rightarrow a X \mid b$
 $X \rightarrow a X \mid b$
 $Y Y \rightarrow A \mid \epsilon$

2.2.5 First and Follow

- First Set $\rightarrow$ extreme Left terminal from which the string of that variable starts.
 $\rightarrow$ it never contains variable, but may contain ' ϵ '.
 $\rightarrow$ we can always find the first of any variable.
- Follow set $\rightarrow$ Follow set contains terminals and \$.
It can never contain variable and ' ϵ '.
How to find follow set?
 1. Include \$ in follow of start variable.
 2. If production is of type $\rightarrow$
 $A \rightarrow \alpha B \beta$ $\alpha \beta \rightarrow$ strings of grammar symbol].
 $\text{follow}(B) = \text{first}(\beta)$.
If, $\beta \rightarrow \epsilon$, i.e. $A \rightarrow \alpha B$, then $\text{follow}(B) = \text{follow}(A)$.
- Productions like: $A \rightarrow \alpha A$ gives no follow set.

2.2.6 Example of first and follow set

- $S \rightarrow AB \mid CD$
 $A \rightarrow aA \mid a$
 $B \rightarrow bB \mid b$
 $C \rightarrow cC \mid c$
 $D \rightarrow dD \mid d$

	First	Follow
S	a, c	\$
A	a	b
B	b	\$
C	c	D
D	D	\$

2.2.7 Entity into Table: Top Down

- No of Rows = number of unique variable in Grammar.
 - No of columns = [Terminals + \$]
 - For a Variable (Row) fill the column (Terminal) if its P, there in its first with the production required.
 - If ϵ is in first put $V \rightarrow \epsilon$ under \$ and its follow.
- If any cell has multiple items, then its not possible to have LL (1) Parser, since that will be ambiguous.
 - In top down we do: Derivation
In Bottom up we do: Production.

Question: Construct LL (1) Parsing Table for the given Grammar: $E \rightarrow E + T \mid T$; $T \rightarrow T * F \mid F$; $F \rightarrow (E) \mid id$; $\Rightarrow G_0$

$\Rightarrow$ Removing Left Recursion:

$E \rightarrow TE'$

$E' \rightarrow + TE' \mid \epsilon$

$T \rightarrow FT' \quad G_1$

$T' \rightarrow * FT' \mid \epsilon$

$F \rightarrow (E) \mid id$

	First	Follow
E	C, id	\$,)
E'	+, ϵ	\$,)
T	C, id	+, \$,)
T'	*, ϵ	+, \$,)
F	C, id	*, +, \$,)

- Left Factoring not Required:
Construction of Table: [LL (1)]

	+	*	(	)	id	\$
E	Error	Error	$E \rightarrow TE'$	Error	$E \rightarrow TE'$	Error
E'	$E' \rightarrow TE'$	Error	Error	$E' \rightarrow \epsilon$	Error	$E' \rightarrow \epsilon$
T	Error	Error	$T \rightarrow FT'$	Error	$T \rightarrow FT'$	error
T'	$T' \rightarrow \epsilon$	$T' \rightarrow * FT'$	Error	$T' \rightarrow \epsilon$	Error	$T' \rightarrow \epsilon$
F	Error	Error	$F \rightarrow (E)$	Error	$F \rightarrow id$	error

- Since for G_1 , Table constructed with no multiple entries. Hence successfully completed. Hence G_1 is LL (1)

Question: Construct LL (1) Parsing Table for the following Grammar:

$S \rightarrow L = R \mid R$; $L \rightarrow * R \mid id$; $R \rightarrow L \Rightarrow G_0$

Solution: Left Factoring:

$$\begin{aligned}
 S &\rightarrow L = R \mid L & S &\rightarrow LX \\
 L &\rightarrow *R \mid id & \Rightarrow & \times \Rightarrow = R \mid \in \\
 R &\rightarrow L & L &\rightarrow *R \mid id \\
 & & R &\rightarrow L
 \end{aligned}
 \Rightarrow G_1$$

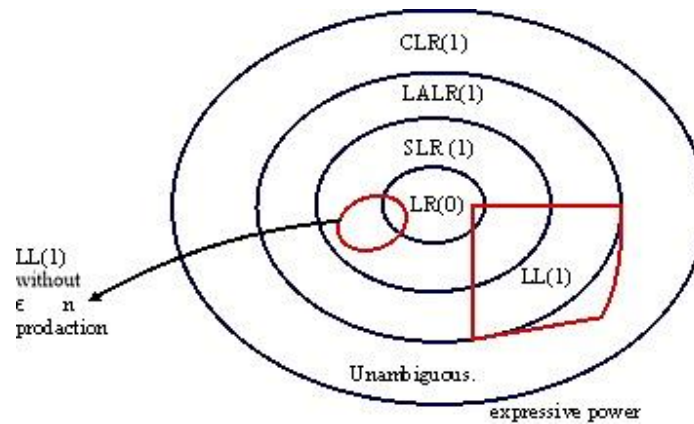
	First	Follow
S	*, id	\$
×	=, ∈	\$
L	*, id	\$
R	*, id	\$

Construction of Table:

	*	=	Id	\$
S	S→LX	Error	S→L X	Error
L	L→*R	Error	L→id	Error
R	R→L	Error	R→L	Error
×	Error	X→ = R	Error	X→∈

- G_1 is a LL (1) Grammar.

2.2.8 Hierarchy of Parsers: [for ϵ - free Grammar]



- For ϵ - producing grammars every LL (1) may not be LALR (1).

Note:

We can't construct any parser for ambiguous grammar. Except: Operator precedence, parser possible for some ambiguous grammar.

Example:

$G: S \rightarrow a S a \mid b S b \mid a \mid b$ (Unambiguous but no parser) $L(G) = \omega(a + b) \omega R$
(Odd palindrome)

- Every RG is not LL (1) as it may be ambiguous, or Recursive or Common Prefix.
- Parsers exists only for the grammar if its Language is DCFL.
- There are some grammars whose Language is DCFL but no parser is possible for it.